# Integración con JavaScript

SP4

**Integrantes:** Acuña Valentín, Rodriguez Gerónimo, Salomon Ezequiel

**Materia:** Aplicaciones Web I

**Profesor:** Andrés Senn

**Carrera:** Análisis de Sistemas

**Turno:** Noche A

## 4.1 Trabajo con eventos JavaScript para filtros, envío de formularios, etc.

El proyecto incorpora JavaScript en las páginas del flujo de reserva: tratamiento, profesional, horario y resumen. Cada una enlaza su propio script mediante la etiqueta script con el atributo type="module".

Se trabaja con el evento change sobre las listas de opciones (radio buttons). El evento se escucha en la lista contenedora (ul) y no en cada opción, ya que se propaga (event bubbling) desde el input marcado hasta la lista. De esta forma se resuelven dos comportamientos:

- **Habilitación del botón:** el botón CONTINUAR inicia deshabilitado (disabled) y se habilita recién cuando se marca una opción. Lo resuelve la función habilitarBoton.
- **Guardado de la elección:** al marcar una opción, su valor se guarda en sessionStorage mediante la función guardarEleccion. evento.target corresponde al radio marcado y .value a su identificador (id del tratamiento, profesional o turno).

El envío de los formularios mantiene el comportamiento nativo del HTML: cada .formulario-seleccion avanza a la página siguiente mediante su atributo action. En esta etapa no se implementan filtros.

## 4.2 Construcción JSON con los datos a renderizar en la interfaz

Los datos que se muestran en la interfaz se organizan en tres catálogos, cada uno como un arreglo de objetos con estructura tipo JSON (pares clave-valor). Cada objeto tiene un id único, que luego se usa como value del radio button y como clave de búsqueda en el resumen.

- **tratamientos** (js/datos/tratamientos.js): 5 objetos con id y nombre.
- **profesionales** (js/datos/profesionales.js): 3 objetos con id, nombre y matrícula.
- **horarios** (js/datos/horarios.js): 3 días, cada uno con un arreglo anidado de turnos (id y hora).

Los catálogos no se guardan en archivos .json, sino en módulos JavaScript que exportan cada arreglo con export const (ver 4.8).

## 4.3 Uso de la función global fetch

En esta etapa no se utiliza la función fetch. Los datos se incorporan con import desde los módulos de js/datos, por lo que el navegador los carga junto con el script de cada página, sin una solicitud aparte (ver 4.8).

## 4.4 Obtener datos desde JSON y renderizar la vista

Las listas de opciones ya no se escriben a mano en el HTML: los ul de tratamiento.html, profesional.html y horario.html quedan vacíos y se completan desde JavaScript. Cada script de listado sigue los mismos pasos:

1. Se importa el catálogo correspondiente desde js/datos.
2. Se selecciona la lista vacía con document.querySelector, a partir de su id (#lista-tratamientos, #lista-profesionales o #lista-horarios).
3. Se recorre el arreglo con forEach y, por cada objeto, se arma con template literals el li con su input radio y su label.
4. Se inserta el HTML generado en la lista mediante innerHTML.

En horarios el recorrido es anidado: un forEach recorre los días y, dentro de cada uno, otro forEach recorre sus turnos. En profesionales se agrega además un span con la clase .matricula.

La página resumen.html realiza el camino inverso. Lee de sessionStorage los id guardados en los pasos anteriores, busca el objeto completo en cada catálogo con el método find y muestra el nombre en los dd correspondientes mediante textContent:

- #resumen-tratamiento: nombre del tratamiento.
- #resumen-profesional: nombre del profesional.
- #resumen-horario: día y hora del turno (por ejemplo, Viernes - 10:00h).

## 4.5 Validaciones del formulario. En el caso de ser necesaria una validación en la introducción de los datos

En los formularios de selección (tratamiento, profesional y horario) se valida que el usuario elija una opción antes de continuar. El botón CONTINUAR permanece deshabilitado hasta que se dispara el evento change (ver 4.1), por lo que no es posible avanzar sin una elección.

En el resumen, si alguno de los datos no se encuentra en sessionStorage, se muestra el texto “No seleccionado” en lugar de dejar el campo vacío.

Los formularios de ingreso de datos (index.html, registrarme.html y recuperarCuenta.html) no cuentan con validaciones por JavaScript en esta etapa. Solo los campos de tipo email aplican la validación nativa del navegador sobre el formato del correo.

## 4.6 Organizar recursos en distintos directorios

Se mantiene la organización de SP3 y se agrega el directorio js, que separa la lógica de los datos:

```
source/
├── *.html           (10 páginas)
├── css/
│   ├── index.css
│   └── fuentes/Lato/
├── img/             (5 íconos .svg)
└── js/
    ├── funciones.js
    ├── listadoTratamientos.js
    ├── listadoProfesionales.js
    ├── listadoHorarios.js
    ├── resumen.js
    └── datos/
        ├── tratamientos.js
        ├── profesionales.js
        └── horarios.js
```

Todas las rutas siguen siendo relativas (./js/listadoTratamientos.js, ./datos/horarios.js), lo que garantiza que el sitio funcione sin importar la ubicación de la carpeta del proyecto.

## 4.7 Modularización del código

El código se modulariza con módulos de JavaScript (ES Modules), mediante las palabras clave export e import. Se divide en tres capas:

- **Datos:** los módulos de js/datos solo exportan los catálogos. Modificar un tratamiento, un profesional o un horario no requiere tocar el HTML ni la lógica.
- **Funciones reutilizables:** js/funciones.js exporta habilitarBoton(lista, boton) y guardarEleccion(lista, clave). Las tres páginas de selección las importan, evitando repetir el mismo código.
- **Scripts de página:** cada página tiene su propio script (listadoTratamientos.js, listadoProfesionales.js, listadoHorarios.js y resumen.js), que solo combina los datos con las funciones.

Al cargarse con type="module", los scripts se ejecutan una vez que el navegador terminó de leer el HTML. Por eso pueden ubicarse en el head y encontrar igualmente las listas con querySelector.

Por último, el sitio debe abrirse desde un servidor local (por ejemplo, Live Server), ya que el navegador bloquea los módulos cuando los archivos se abren directamente (file://).

## 4.8 Cambios con respecto a la propuesta

Respecto de lo planteado en las consignas, se registran los siguientes cambios:

- **Datos en módulos JavaScript en lugar de archivos JSON:** los catálogos se definen como arreglos exportados desde js/datos y se incorporan con import. Por este motivo no se emplea fetch para obtenerlos.
- **Filtros:** no se implementan en esta etapa. Los eventos se aplican a la selección de opciones y al guardado de la elección.
- **Validaciones:** se limitan a los formularios de selección y al resumen. Los formularios de ingreso de datos quedan sin validación por JavaScript.
- **Confirmación del turno:** el botón de resumen.html (envío por WhatsApp y evento en Google Calendar) se mantiene visible pero sin funcionalidad.